

Harvard Academia Atlas

Raima Islam, Kumar Tanmay, Aditya Saxena

Problem Statement:

The Harvard Academia Atlas helps students navigate their academic journey by offering personalized course recommendations based on their backgrounds, interests, and goals. It suggests courses, explains their relevance to academic and career paths and helps create conflict-free schedules for easy management.

Background and Motivation:

Course selection from Crimson Cart is stressful for Harvard students, who must juggle heavy workloads, conflicting schedules, and the need for relevant academic experience. Many also balance research, part-time jobs, and extracurriculars, complicating academic planning. To address this, we developed a Generative AI-powered tool to simplify the process by offering tailored course suggestions and conflict-free schedules.

Objectives:

The goal is to develop a web application with a chat-based interface that helps users plan courses based on their academic and career goals while avoiding timetable conflicts. It will offer personalized course recommendations, explain why specific courses are valuable, and generate conflict-free schedules.

Source of Data:

We have open-source course data from Harvard Law School and Harvard Business School and are focusing on students in these schools. We will only use publicly available data and will not access any restricted information without permission.

Description of Dataset:

The Harvard Course Catalog is a comprehensive resource that helps students select courses. It provides details like class codes, descriptions, professor information, permissions, petitions, cross-registration options, schedules, availability, and prerequisites.

Key Attributes:

- **Course Description:** Summarizes objectives, topics, and theory-practice balance to aid academic and career alignment.
- **Instructor:** Reputation, teaching style, and research focus are key factors in course selection.
- **Class Codes:** Help identify courses and track prerequisites or levels.
- **Prerequisites:** Ensure students meet the required background before enrolling.
- **Class Availability:** Affects course planning based on open spots.
- **Cross-Registration:** Allows enrollment in courses at other Harvard or non-Harvard institutions.
- **Permissions and Petitions:** Needed for limited-space or specific-entry courses.
- **Class Timings:** Influence course prioritization due to scheduling conflicts.
- **Workload Balance:** Helps students balance academics with other responsibilities.

Relevance to the Project:

The Harvard Course Catalog is vital for our course-planning tool, offering key details like course descriptions, instructor info, and schedules. This data enables tailored, conflict-free recommendations based on student goals.

Data Quality Concerns:

- **Missing Data:** Incomplete course details can impact recommendation accuracy.
- **Inconsistent Data:** Variations in formatting across departments may cause inconsistencies.
- **Dataset Combination:** Merging data from the course catalog and schedules could reduce accuracy.
- **Imbalanced Q-Report Data:** Popular courses have more feedback, while lesser-known courses may need more data, leading to biased recommendations.

Minimum Components for a Good Project:

- **Large or Heterogeneous Data:** Diverse datasets, including academic records and student preferences, will be managed using TensorFlow Datasets and PyTorch DataLoader.
- **Scalability:** Cloud infrastructure like Kubernetes, Vertex AI, and CI/CD pipelines will ensure efficient scalability for large user bases.
- **Complex Models:** Large language models (LLMs) will be fine-tuned with Hugging Face, LangChain, and LORA, utilizing GPU compute for real-world optimization.
- **Efficient Inference:** Model compression and distillation techniques will reduce the computational cost of running fine-tuned LLMs at scale.

Learning Emphasis:

- **Frontend:** HTML, CSS, Django, and JavaScript will create a responsive, user-friendly interface.
- **Database:** NoSQL and relational databases will handle structured and unstructured data.
- **Language:** Python will power backend development and model training due to its versatility.
- **Tech Stack:** Vector databases, FastAPI, Hugging Face, OpenAI, and fine-tuned LLMs will ensure scalability.
- **Cloud:** AWS, GCP, Kubernetes, MLOps, and Kubeflow will support scalable deployment and workflows.

Research and Development:

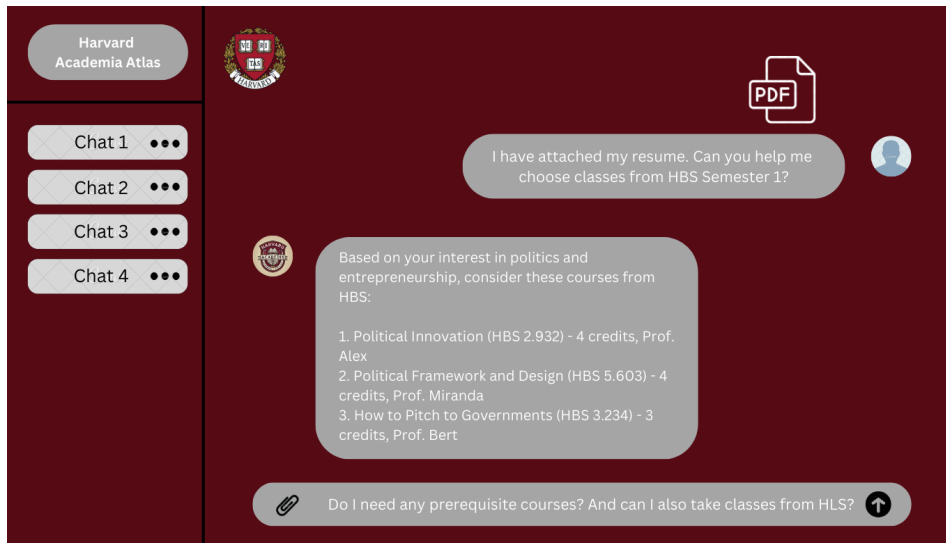
[1] <https://iopscience.iop.org/article/10.1088/1757-899X/1098/3/032039/pdf>

[2] <https://medium.com/@sahin.samia/what-is-retrieval-augmented-generation-rag-in-llm-and-how-it-works-a8c79e35a172>

[3] <https://www.thinkingstack.ai/blog/operationalisation-1/scalability-in-mlops-handling-large-scale-machine-learning-models-15>

[4] <https://huggingface.co/docs/transformers/en/training>

Application Mock Design:



Fun Factor:

This acts as your course instructor buddy therefore save yourself the headache and use our AI-assisted tool to plan your dream schedule. Because either way, at Harvard, who gets time to even overthink? Let our app do the overthinking for you. Plus, who wouldn't want a little AI sidekick to help plan their semester?

Limitations and Risks:

Challenges include ensuring high-quality data for accurate recommendations and managing the complexity of fine-tuning a large language model. Scaling to support a growing user base while maintaining a responsive chat interface is another difficulty. Fine-tuning requires significant GPU resources, and large language models have limited context length, making dataset preparation critical. There's also the risk of generating biased or incorrect outputs, which could impact user experience.

Milestones:

- 1. MS1 (09/20, 4%):** Project Proposal & Team Formation – Finalize the project idea, form the team, submit the proposal, and receive feedback and approval.
- 2. MS2 (10/18, 10%):** Initial Setup & Infrastructure – Set up the development environment and foundational MLOps infrastructure, including containers and data pipelines.
- 3. MS3 (10/31, 25%):** Midterm Check-in – Present a basic version with a simple working model, such as a CLI demonstrating course recommendations or scheduling conflict resolution.
- 4. MS4 (11/15, 14%):** Frontend & API Integration – Develop the frontend with basic functionality, ensuring it integrates with the APIs for intuitive user interaction.
- 5. MS5 (12/11, 35%):** Final Presentation & Deployment – Polish the frontend and backend, create a Docker container, deploy on Kubernetes, prepare documentation, and present via a short video and GitHub repository.